DIGITAL LOGIC DESIGN

CSE 1325 · Course-Outline Study Notes | UIU Spring 2026

Topics follow the supplied outline’s lecture sequence. Broad syllabus topics not individually mapped to the 24-class outline are identified separately.

1. Digital Logic, Number Systems and Conversion

Digital circuits represent information with discrete values, commonly 0 and 1. The base (radix) determines available digits and positional weights.

System	Base	Digits	Example
Binary	2	0, 1	$(1011)_2 = 11_{10}$
Octal	8	0–7	$(17)_8 = 15_{10}$
Decimal	10	0–9	$(25)_{10}$
Hexadecimal	16	0–9, A–F	$(2F)_{16} = 47_{10}$

$$(d_k d_{k-1} \ldots d_0)_b = \sum_{i=0}^{k} d_i b^i$$

Positional value: digit × base raised to its position

$$(101101)_2 = 1(2^5) + 1(2^3) + 1(2^2) + 1(2^0) = 45$$

Binary-to-decimal example

Decimal → binary: repeatedly divide by 2 and read remainders bottom-up. Binary → decimal: add powers of 2 for bits equal to 1. Fractional binary places use 2^{-1}, 2^{-2}, etc.

2. BCD, Binary Arithmetic and Complements

Addition: 0+0=0, 0+1=1, 1+0=1, 1+1=10_2. Subtraction: 0−1 requires borrowing. Multiplication uses shifted partial products. BCD encodes each decimal digit separately in four bits; e.g. 59 → 0101 1001.

BCD correction: if a four-bit sum exceeds 1001_2 (9), or a carry occurs, add 0110_2 (6). Example: 7+5 gives 1100_2; add 0110_2 → 1 0010, representing BCD 0001 0010 (decimal 12).

$$\text{Two's complement} = \text{one's complement} + 1$$

For fixed-width subtraction, add the two’s complement of the subtrahend.

3. Gray Code and Alphanumeric Codes

$$G_{\text{MSB}} = B_{\text{MSB}}, \qquad G_i = B_{i+1} \oplus B_i$$

Binary to Gray: preserve the most significant bit; XOR adjacent binary bits.

$$B_i = B_{i+1} \oplus G_i$$

Gray to binary: decode cumulatively from the most significant bit.

Decimal	Binary	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

ASCII is a widely used character encoding. Letters, digits and symbols are represented by bit patterns; interpretation depends on the selected encoding.

4. Logic Gates, Truth Tables and Boolean Algebra

Basic logic-gate truth table

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Illustrative diagram for learning; actual circuit/function depends on the problem statement.

$$Y = A \cdot B \text{ (AND)}, \quad Y = A + B \text{ (OR)}, \quad Y = \overline{A} \text{ (NOT)}$$

$$Y = A \oplus B \text{ (XOR)}$$

Identity	Boolean rule
Identity	A+0=A; A·1=A
Domination	A+1=1; A·0=0
Idempotent	A+A=A; A·A=A
Complement	A+Ā=1; A·Ā=0
Commutative	A+B=B+A; AB=BA
Associative	A+(B+C)=(A+B)+C; A(BC)=(AB)C
Distributive	A(B+C)=AB+AC; A+BC=(A+B)(A+C)
Absorption	A+AB=A; A(A+B)=A

De Morgan’s laws

$$\overline{AB} = \overline{A} + \overline{B}, \qquad \overline{A + B} = \overline{A}\,\overline{B}$$

Duality swaps + with · and 0 with 1, leaving variables and complements unchanged. From a truth table, rows with output 1 produce SOP; rows with output 0 produce POS. Complement a function by reversing its output values or negating its expression.

5. Minterms, Maxterms, K-Maps and Minimization

Term	Meaning	Example (A,B)
Minterm	AND of each variable once; equals 1 for one row.	m_1=ĀB for input 01
Maxterm	OR of each variable once; equals 0 for one row.	M_1=A+B̄ for input 01

F = Σm(indices where F=1); F = ΠM(indices where F=0). Indices follow the binary ordering of inputs. SOP is an OR of product terms; POS is an AND of sum terms.

Illustrative 3-variable Karnaugh map

AB/C	0	1
00	0	1
01	1	0
11	1	1
10	0	1

Illustrative diagram for learning; actual circuit/function depends on the problem statement.

K-map method: place outputs in Gray-code order; group adjacent 1s in powers of two for SOP or 0s for POS. Use the largest groups; edges wrap around and overlap is allowed if useful. Variables that change within a group are eliminated. Don’t-care entries may be 0 or 1 when they simplify the function.

$$F = AB + \overline{A}C \quad \text{(SOP)}, \qquad F = (A + B)(\overline{A} + C) \quad \text{(POS)}$$

Example forms

Canonical SOP/POS expands terms until each includes every input variable. Verify equivalence with a truth table. Circuit cost may mean gate count, gate inputs, literals or logic levels—use the criterion specified by the problem.

6. Combinational Circuit Design

Design sequence: identify inputs/outputs → make the truth table → derive Boolean functions → simplify with identities/K-maps → implement using gates or blocks → verify all input combinations.

Block	Function
Decoder	n-bit code activates one of up to 2^n output lines.
Encoder	Converts one active input to a binary code; a priority encoder resolves multiple active inputs.
MUX	Selects one of several data inputs; a 2^n-to-1 MUX uses n select bits.
DEMUX	Routes one input to one of several outputs; a 1-to-2^n DEMUX uses n select bits.

Half adder: S=A⊕B, C=AB. Full adder:

$$S = A \oplus B \oplus C_{\text{in}}$$

Sum

$$C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$$

Carry out

Ripple-carry adders pass carry through successive stages, so delay accumulates. Faster adder structures aim to reduce carry propagation delay.

7. Sequential Circuits and Sequence Recognizers

Feature	Combinational	Sequential
Depends on	Current inputs	Inputs and stored state
Memory	No stored state required	Latches/flip-flops
Examples	Adder, decoder, MUX	Register, counter, sequence recognizer

A sequence recognizer detects a specified input pattern. Design: define states → state diagram → state table → binary assignment → next-state/output equations → flip-flop implementation.

8. Buffers, Latches and Flip-Flops

Element	Core idea
Buffer	Passes/strengthens a signal; tri-state buffer can output high impedance Z.
SR latch	Set/Reset storage; invalid condition depends on NOR/NAND implementation.
D latch	When enabled, output follows D; otherwise holds its state.
D flip-flop	Samples D at its active clock edge.
JK flip-flop	J=K=1 toggles state at the active clock edge.
T flip-flop	T=1 toggles; T=0 holds.

$$Q^+ = D \quad \text{(D flip}-\text{flop)}$$

$$Q^+ = J\overline{Q} + \overline{K}Q \quad \text{(JK flip}-\text{flop)}$$

$$Q^+ = T \oplus Q \quad \text{(T flip}-\text{flop)}$$

Master-slave flip-flops use two stages on opposite clock phases. Race-around can occur in a level-triggered JK flip-flop when J=K=1 and the active clock pulse is too long, allowing repeated toggles.

9. Registers and Counters

Parallel-load registers load multiple bits together. Shift registers move bits left/right; types include SISO, SIPO, PISO and PIPO. Asynchronous/ripple counters clock later stages from earlier outputs, so delays accumulate; synchronous counters share a clock. Up counters increase in binary; down counters decrease.

Number of possible states with n bits $= 2^n$

Illustrative 2-bit up-counter: 00 → 01 → 10 → 11 → 00



Illustrative diagram for learning; actual circuit/function depends on the problem statement.

A modulo-M counter has M distinct states before repeating. Design the sequence, state table and next-state equations, then verify unused states.

10. Other Topics Listed in the Syllabus

These topics are listed in the broad syllabus but are not individually mapped to the supplied 24-class lecture outline; this is a checklist, not a claim of detailed lecture coverage.

Topic	Brief meaning
PLA	Programmable AND/OR planes implement Boolean functions.
Mealy / Moore machines	Mealy output depends on state and input; Moore output depends on state.
State minimization/assignment	Merge equivalent states when valid and assign binary codes.
Pulse-mode / fundamental-mode	Asynchronous sequential design approaches.
Diode logic / transistor switches	Components used for switching and simple logic.
TTL, ECL, IIL, CMOS	Logic families with different speed, power and voltage characteristics.
Delay, power, noise immunity	Timing, energy use, logic margins and noise resistance.
Open-collector / high-impedance gates	Pull-ups may be needed for open-collector outputs; tri-state outputs can disconnect.
Memory systems	Organization, addressing, read/write behavior.
A/D and D/A converters	Conversion between analog signals and digital codes.

Quick Revision

Topic	Key idea / formula
Positional value	$\Sigma\, d_i b^i$
Two’s complement	Invert bits and add 1 at fixed width
Gray code	GMSB=BMSB; $G_i = B_{i+1} \oplus B_i$
De Morgan	overline(AB)=Ā+B̄; overline(A+B)=ĀB̄
SOP / POS	OR of products / AND of sums
Half adder	S=A⊕B; C=AB
Full adder	S=A⊕B⊕Cin; Cout=AB+ACin+BCin
n-bit state space	2^n states
MUX / DEMUX	Many inputs→one output / one input→selected output